Parallelising

Self Organising Map

A really bad idea I somehow convinced my teacher to let me try

Introduction:

A Self Organizing Map (SOM) is a heuristic clustering algorythm. “In its basic form it produces a similiraty graph of input data.“(1.) To this extent, I have implemented it with so called K-Center clustering, where each neuron or weight approximates the center of clusters of data points. It achieves this by first deciding which neighbours a neuron has in euclidian distance. The number of neighbours is decided at program launch. We omit using the square root for computational savings for each euclidean calculation, since we don't need the exact distance, only the minimum.

$$\sum_{0}^{dim} weights[d][i] - weights[d][j]$$

Where dim is the number of dimensions, d the current dimension and i the neuron we want to know the distance of to neuron j.
Then for every point we find the euclidean distance to its closest neuron from point x:

$$\sum_{0}^{dim} weights[d][i] - x[d]$$

The closest neuron to x is moved toward it using the formula:

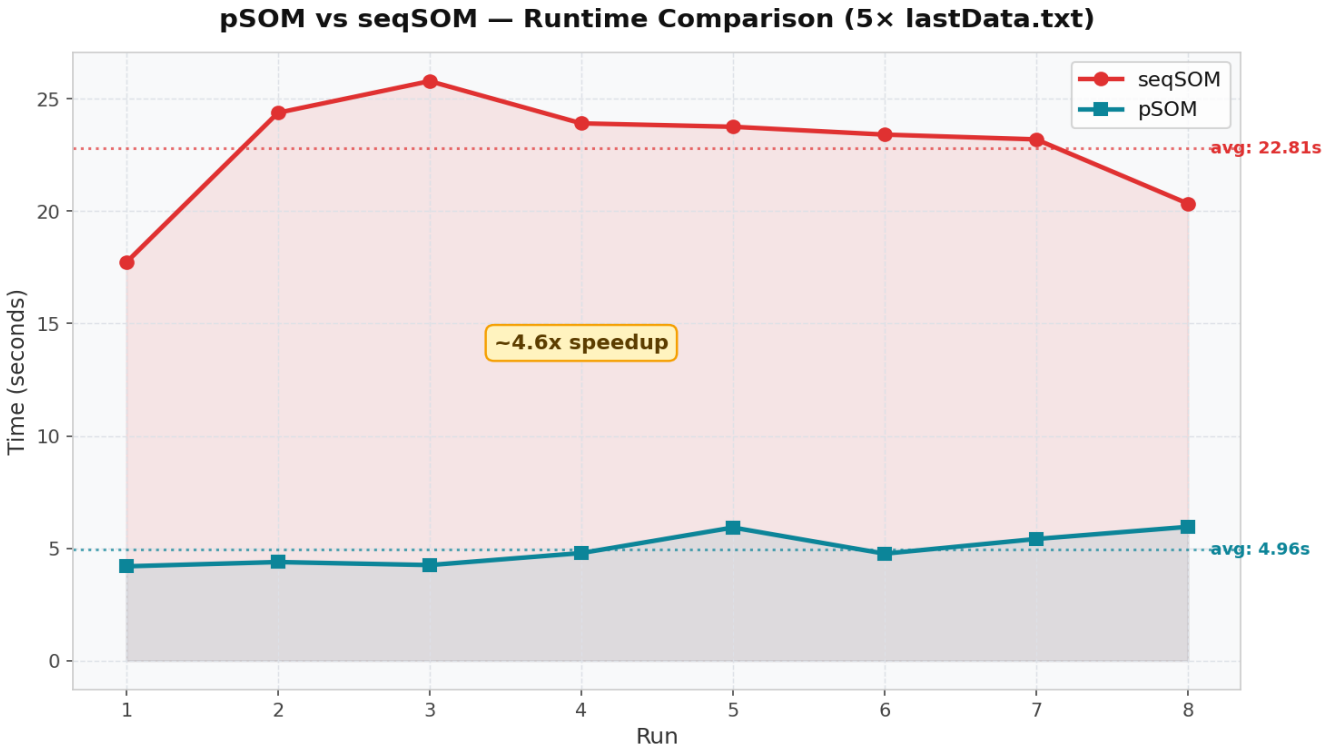
$$\sum_{0}^{dim} l\left(x[d] - weights[d][idx]\right)$$

Where idx is the id of the neuron being updated and l is the learning rate, decided at program launch.
Every few epochs, according to a parameter set at launch, all neighbours of each neuron get updated for each point where the learning rate is multiplied with another parameter set at launch, the influence. After all epochs have run, we find the closest neuron to each point and finally the program returns an array where all points have clustered under a common neuron.

Parallelisation:

Of the 12 attempts to achieve speedup, 4 worked, 2 are not related to parallelisation and the remaining 2 are incompatible with each other. This is due to the very sequential nature of a SOM, where in each epoch, for each data point, neurons get updated. We cannot parallelise any of these three loops, since each iteration depends on the result of the last.
Parallelising the euclidean distance calculations (EDC) resulted in a slowdown each time. This is due to the loops iterating over dimensions, which rarely pass 1000. Also there needs to be a reduction, which adds additional overhead. Calling these loops inside the necessarily sequential epoch and data point loops mentioned earlier happens too often for too little time. In tests parallel EDC ran about 283025 ms, sequential EUC ran 42.3214 ms, on average. There was only one sample from the parallel EUC, due to the great discrepancy of the sequential EDC being about 6700 times faster.
Parallelising the neighbour finding function incurred too much overhead, since the number of neurons rarely reaches three digits and the number of neighbours is defined as a parameter, usually single digit, since more ruin the SOM. Too many neighbours influence single neurons too much to approximate any graph.
Parallelising the neighbour update function was impossible due to the inherent sequentiality of updating neurons.

Transposing the weights matrix from neurons × dimensions to dimensions × neurons gave a speedup. Since it is not a parallel speedup, testing was omitted.
Removing the root pulling from the euclidean distance functions resulted in not measurable speedup.
Initialising the weights matrix could be parallelised, resulting in a speedup of 16.6. Using a custom reduction, the minimum and maximum of each point in each dimension were used to find the average across the entire data set.
This is inherently incompatible with the main parallelisation settled on at the end of this venture.
Since a SOM is a heuristic algorythm, a parallel approach to the same data with differing parameters may result in more favorable mapping. To this end, my program can be input with multiple files, with the ability to set parameters for each.



lastData.txt was created with 10 cluster centers, 10 dimensions and 200000 points

Parallel execution allowed pSOM to finish the same data set 4.6 times faster. It may be noted, that the sequential execution from seqSOM here would save about 2s if, for example, run on different machines at the same time. Overhead from parallelisation is to blame.

Conclusion:

Learning from mistakes is the most fruitful way to understand any topic. This assignment taught me how to apply various aspects of parallelisation to the presented code and what was incompatible with the algorythm(2.). It gave me a greater understanding on how parallelisation works by finding many ways it failed due to inherent sequentiality. Going forth, I will have little difficulty discerning what algorythm can be parallelised nicely, due to having run into so many dead ends during this assignment.
Parallelising a SOM can only be done by running multiple instances in parallel. The initialisation process can be parallelised, but must be disjointed from training.

Usage of GPT:

Since the code this Assignment stands on was written in Python, I decided to use GPTs like ChatGPT and Claude to translate it for me into C++, to save time. My understanding of the pseudocode was crucial, since both made mistakes I had to reconcile manually. Following the translation, I verified the Openmp code I used with a GPT, after I did not achieve any speedup. I had it explain to me what was going wrong, but I would only truly understand after sketching sections I had parallelised on paper or whiteboard. At this point I must also give thanks to the genuine Intelligences that were my lecturer Fredrik Manne and tutoring assistant Juni Weisteen Bjerde.
To save time timing my code, I used GPT. I confirmed by going through the code myself that it was timing what I wanted it to. Almost all print statements were done by GPT. Transposing the weights matrix and subsequent EDC was done by a GPT, since it had no greater bearing on parallelisation. It was part of an attempt at parallelisation, which turned out to not work. Cache lines were not the issue when the loops were smaller than triple digit iterations. To understand custom reductions I watched a Youtube video(3.) and had my implementation attempt explained to me by GPT, which corrected my approach, that I ended up using.
Data loading was coded by GPT, since it was irrelevant to the task at hand and saved me time.

References:

(1.)Teuvo Kohonen, “Self-Organizing Maps Third Edition “ in Springer Series in Information Sciences 30 https://link.springer.com/book/10.1007/978-3-642-56927-2

(2.) https://github.com/Robsel/C-C-Sesselpupser/tree/C-Implementation

(3.)https://www.youtube.com/watch?v=gW9EiEQAkDU